20

Rendering Item Lists

In this chapter, you'll learn how to work with item lists. Lists are a common web application component. While it's relatively straightforward to build lists using the `<ul>` and `<li>` elements, doing something similar on native mobile platforms is much more involved.

Thankfully, React Native provides an **item list** interface that hides all of the complexity. First, you'll get a feel for how item lists work by walking through an example. Then, you'll learn how to build controls that change the data displayed in lists. Lastly, you'll see a couple of examples that fetch items from the network.

We'll cover the following topics in this chapter:

- Rendering data collections
- Sorting and filtering lists
- Fetching list data
- Lazy list loading

- Implementing pull to refresh

Technical requirements

You can find the code files for this chapter on GitHub at https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter20.

Rendering data collections

Lists are the most common way to display a lot of information: for example, you can display your friend list, messages, and news. Many apps contain lists with data collections, and React Native provides the tools to create these components.

Let's start with an example. The React Native component you'll use to render lists is `FlatList`, which works the same way on iOS and Android. List views accept a `data` property, which is an array of objects. These objects can have any properties you like, but they do require a `key` property. If you don't have a `key` property, you can pass the `keyExtractor` prop to the `Flatlist` component and instruct what to use instead of `key`. The `key` property is similar to the requirement for rendering the `<li>` elements inside of a `<ul>` element. This helps the list to efficiently render when changes are made to list data.

Let's implement a basic list now. Here's the code to render a basic 100-item list:

```
const data = new Array(100)
  .fill(null)
  .map((v, i) => ({ key: i.toString(), value: `Item ${i}` }));
export default function App() {
  return (
    <View style={styles.container}>
      <FlatList
        data={data}
        renderItem={({ item }) => <Text style={styles.item}>{item.value}</Text>}
      />
    </View>
  );
}
```

Let's walk through what's going on here, starting with the `data` constant. This has an array of 100 items in it. It is created by filling in a new array with 100 `null` values and then mapping this to a new array with the objects that you want to pass to `<FlatList>`. Each object has a `key` property because this is a requirement; anything else is optional. In this case, you've decided to add a `value` property that will be used later when the list is rendered.

Next, you render the `<FlatList>` component. It's within a `<View>` container because list views need height in order to make scrolling work correctly. The `data` and `renderItem` properties are passed to `<FlatList>`, which ultimately determines the rendered content.

At first glance, it seems like the `FlatList` component doesn't do too much. Do you have to figure out how the items look? Well, yes, the `FlatList` component is supposed to be generic. It's supposed to excel at handling updates and embeds scrolling capabilities into lists for us. Here are the styles that were used to render the list:

```
import { StyleSheet } from "react-native";
export default StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    paddingTop: 40,
  },
  item: {
    margin: 5,
    padding: 5,
    color: "slategrey",
    backgroundColor: "ghostwhite",
    textAlign: "center",
  },
});
```

Here, you're styling each item on your list. Otherwise, each item would be text-only, and it would be difficult to differentiate between other list items. The `container` style gives the list height by setting `flex` to `1`.

Let's see what the list looks like now:

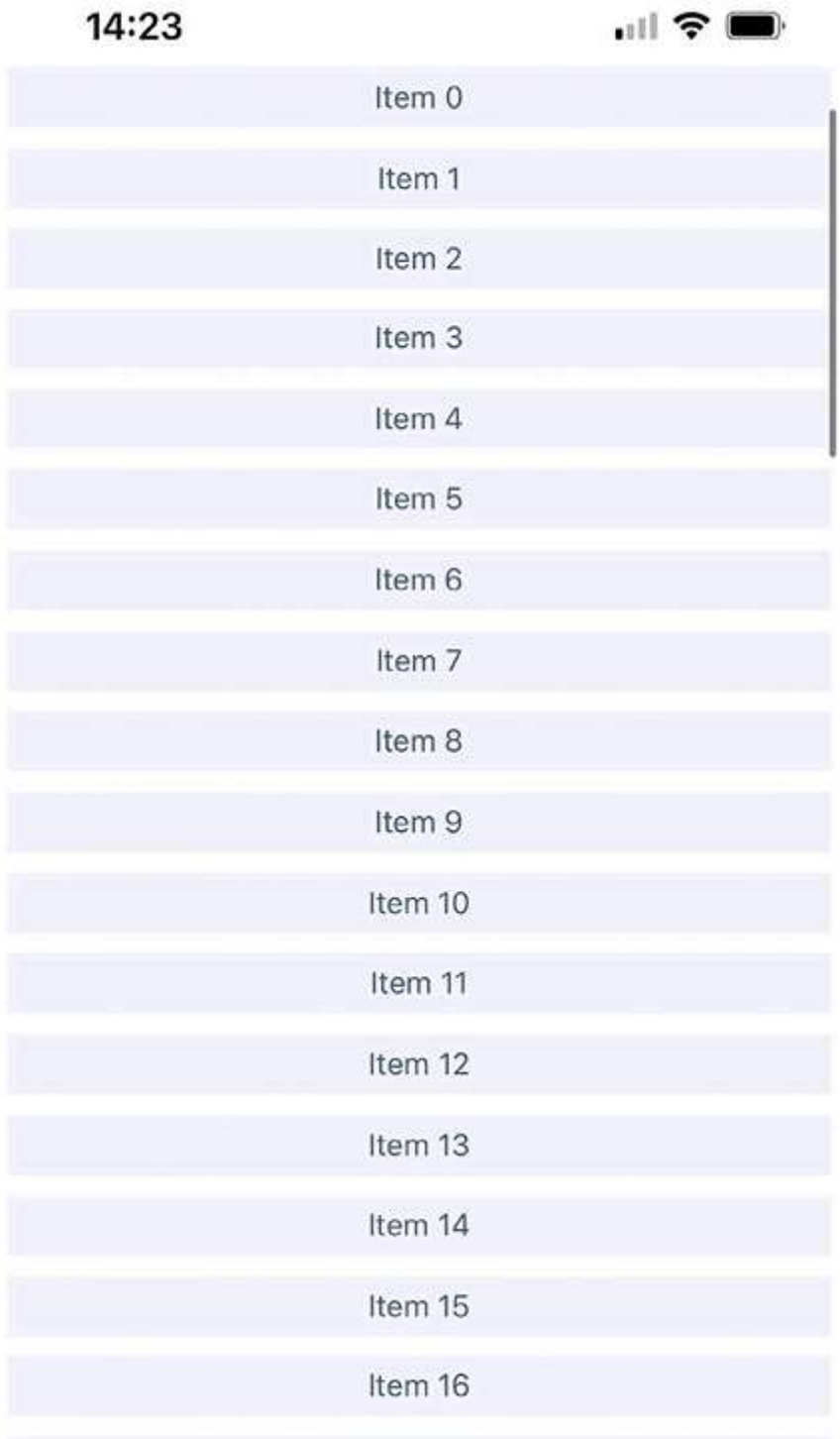

14:23
Item 0
Item 1
Item 2
Item 3
Item 4
Item 5
Item 6
Item 7
Item 8
Item 9
Item 10
Item 11
Item 12
Item 13
Item 14
Item 15
Item 16

Figure 20.1: Rendering the data collection

If you're running this example in a simulator, you can click and hold down the mouse button anywhere on the screen, like a finger, and then scroll up and down through the items.

In the following section, you'll learn how to add controls for sorting and filtering lists.

Sorting and filtering lists

Now that you have learned the basics of the `FlatList` components, including how to pass data, let's add some controls to the list that you just implemented in the *Rendering data collections* section. The **FlatList** component can be rendered together with other components: for example, list controls. It helps you to manipulate the data source, which ultimately drives what's rendered on the screen.

Before implementing list control components, it might be helpful to review the high-level structure of these components so that the code has more context. Here's an illustration of the component structure that you're going to implement:

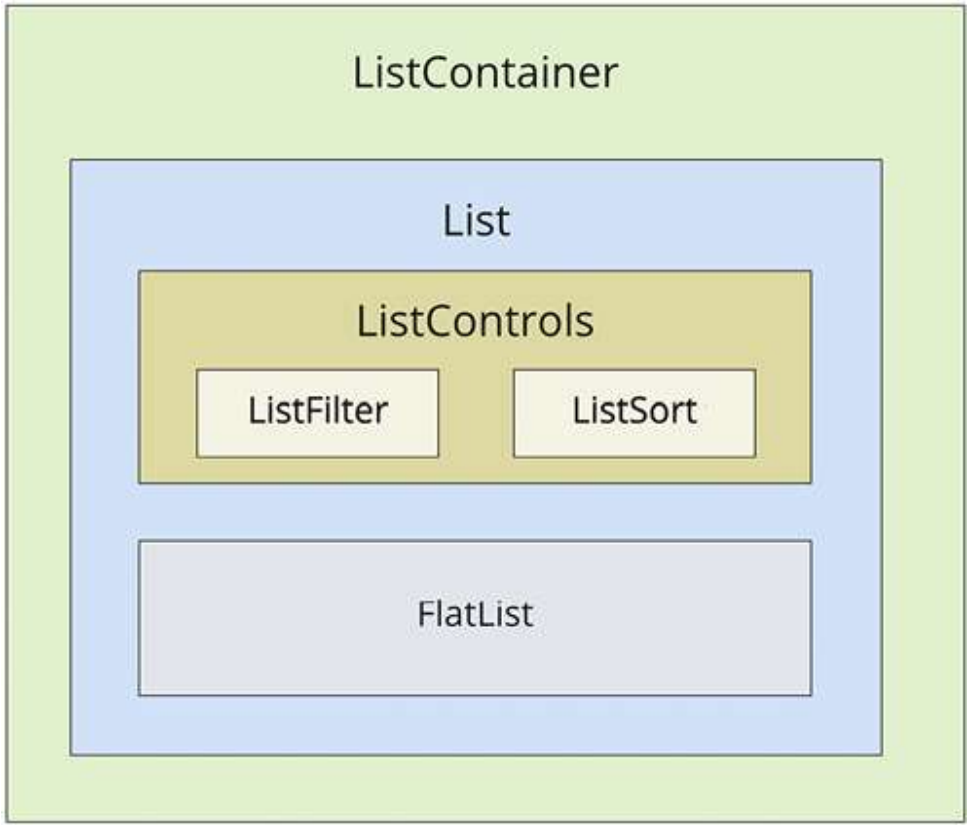


Figure 20.2: The component structure

Here's what each of these components is responsible for:

- `ListContainer`: The overall container for the list; it follows the familiar React container pattern
- `List`: A stateless component that passes the relevant pieces of state into `ListControls` and the React Native `ListView` component
- `ListControls`: A component that holds the various controls that change the state of the list
- `ListFilter`: A control for filtering the item list
- `ListSort`: A control for changing the sort order of the list
- `FlatList`: The actual React Native component that renders items

In some cases, splitting apart the implementation of a list like this is overhead. However, I think that if your list needs controls in the first place, you're probably implementing something that will stand to benefit from having a well-thought-out component architecture.

Now, let's drill down into the implementation of this list, starting with the **ListContainer** component:

```
function mapItems(items: string[]) {
  return items.map((value, i) => ({ key: i.toString(), value }));
}
const array = new Array(100).fill(null).map((v, i) => `Item ${i}`);
function filterAndSort(text: string, asc: boolean): string[] {
  return array
```

```
      .filter((i) => text.length === 0 || i.includes(text))
      .sort(
        asc
          ? (a, b) => (a > b ? 1 : a < b ? -1 : 0)
          : (a, b) => (b > a ? 1 : b < a ? -1 : 0)
      );
  }
```

Here, we define a few utility functions and the initial array that we will use.

Then, we will define `asc` and `filter` for managing sorting and filtering the list, respectively, with the `data` variable implemented using the `useMemo` hook:

```
export default function ListContainer() {
  const [asc, setAsc] = useState(true);
  const [filter, setFilter] = useState("");
  const data = useMemo(() => {
    return filterAndSort(filter, asc);
  }, [filter, asc]);
```

It gives us an opportunity to avoid updating it manually because it will be recalculated automatically when the `filter` and `asc` dependencies are updated. It also helps us to avoid unnecessary recalculation when `filter` and `asc` are not changed.

This is how we apply this logic to the `List` component:

```
return (
  <List
    data={mapItems(data)}
    asc={asc}
    onFilter={(text) => {
      setFilter(text);
    }}
    onSort={() => {
      setAsc(!asc);
    }}
  />
);
```

If this seems like a bit much, it's because it is. This container component has a lot of state to handle. It also has some non-trivial behavior that it needs to make available to its children. If you look at it from the perspective of an encapsulating state, it will be more approachable. Its job is to populate the list with state data and provide functions that operate in this state.

In an ideal world, the child components of this container should be nice and simple, since they don't have to directly interface with the state. Let's take a look at the `List` component next:

```
export default function List({ data, ...props }: Props) {
  return (
    <FlatList
      data={data}
      ListHeaderComponent={<ListControls {...props}/>}
      renderItem={({ item }) => <Text style={styles.item}>{item.value}</Text>}
    />
  );
}
```

This component takes the state from the `ListContainer` component as a property and renders a `FlatList` component. The main difference here from the previous example is the `ListHeaderComponent` property. This renders the controls for your `List` component. What's especially useful about this property is that it renders the controls outside the scrollable list content, ensuring that the controls are always visible. Let's take a look at the `ListControls` component next:

```
type Props = {
  onFilter: (text: string) => void;
  onSort: () => void;
  asc: boolean;
};
export default function ListControls({ onFilter, onSort, asc }: Props) {
  return (
    <View style={styles.controls}>
      <ListFilter onFilter={onFilter} />
```

```
        <ListSort onSort={onSort} asc={asc} />
      </View>
    );
  }
```

This component brings together the `ListFilter` and `ListSort` controls. So, if you were to add another list control, you would add it here.

Let's take a look at the `ListFilter` implementation now:

```
type Props = {
  onFilter: (text: string) => void;
};
export default function ListFilter({ onFilter }: Props) {
  return (
    <View>
      <TextInput
        autoFocus
        placeholder="Search"
        style={styles.filter}
        onChangeText={onFilter}
      />
    </View>
  );
}
```

The filter control is a simple text input that filters the list of items by user type. The `onFilter` function that handles this comes from the `ListContainer` component.

Let's look at the `ListSort` component next:

```
const arrows = new Map([
  [true, "▼"],
  [false, "▲"],
]);
type Props = {
  onSort: () => void;
  asc: boolean;
};
export default function ListSort({ onSort, asc }: Props) {
  return <Text onPress={onSort}>{arrows.get(asc)}</Text>;
}
```

Here's a look at the resulting list:

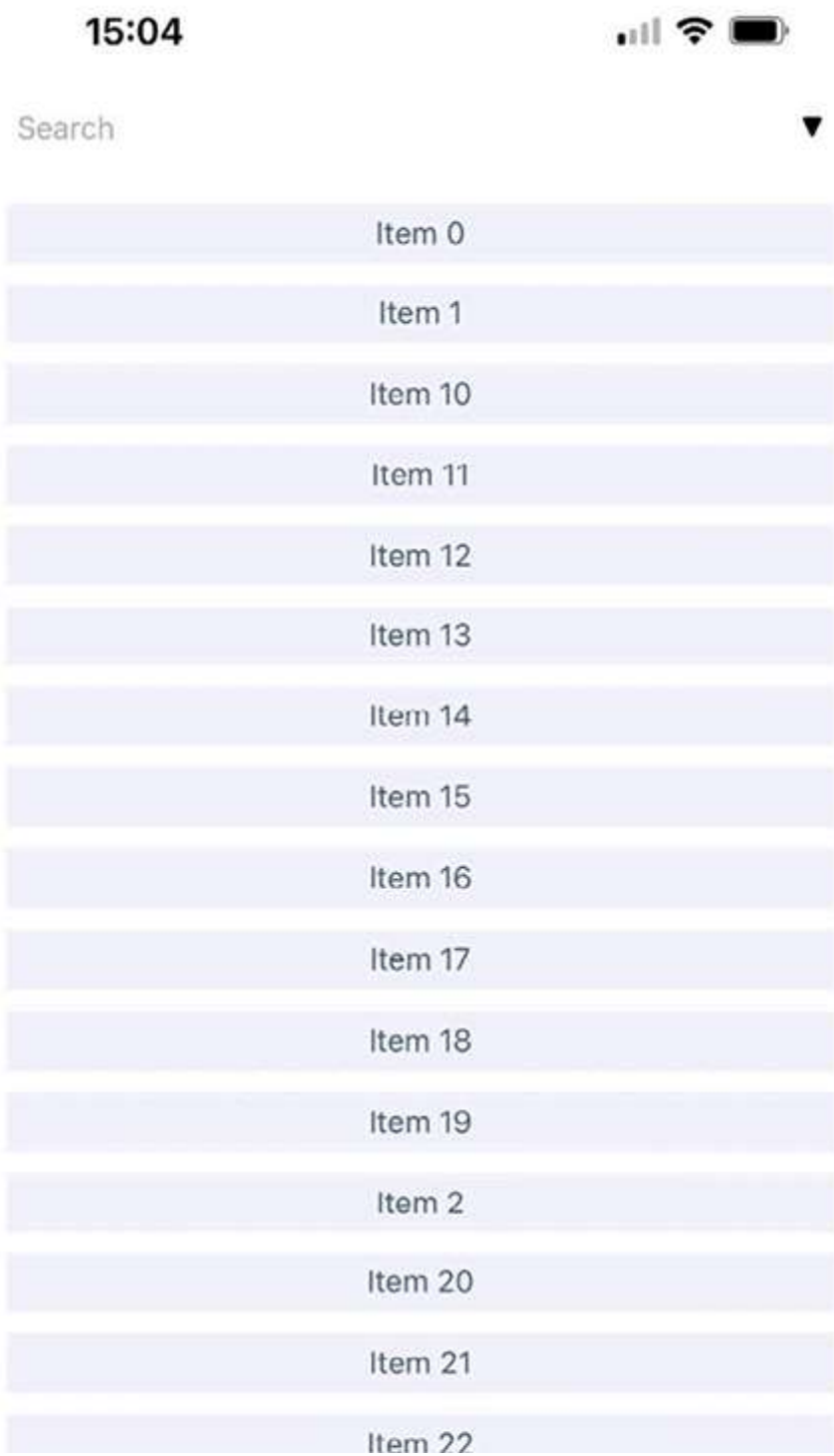
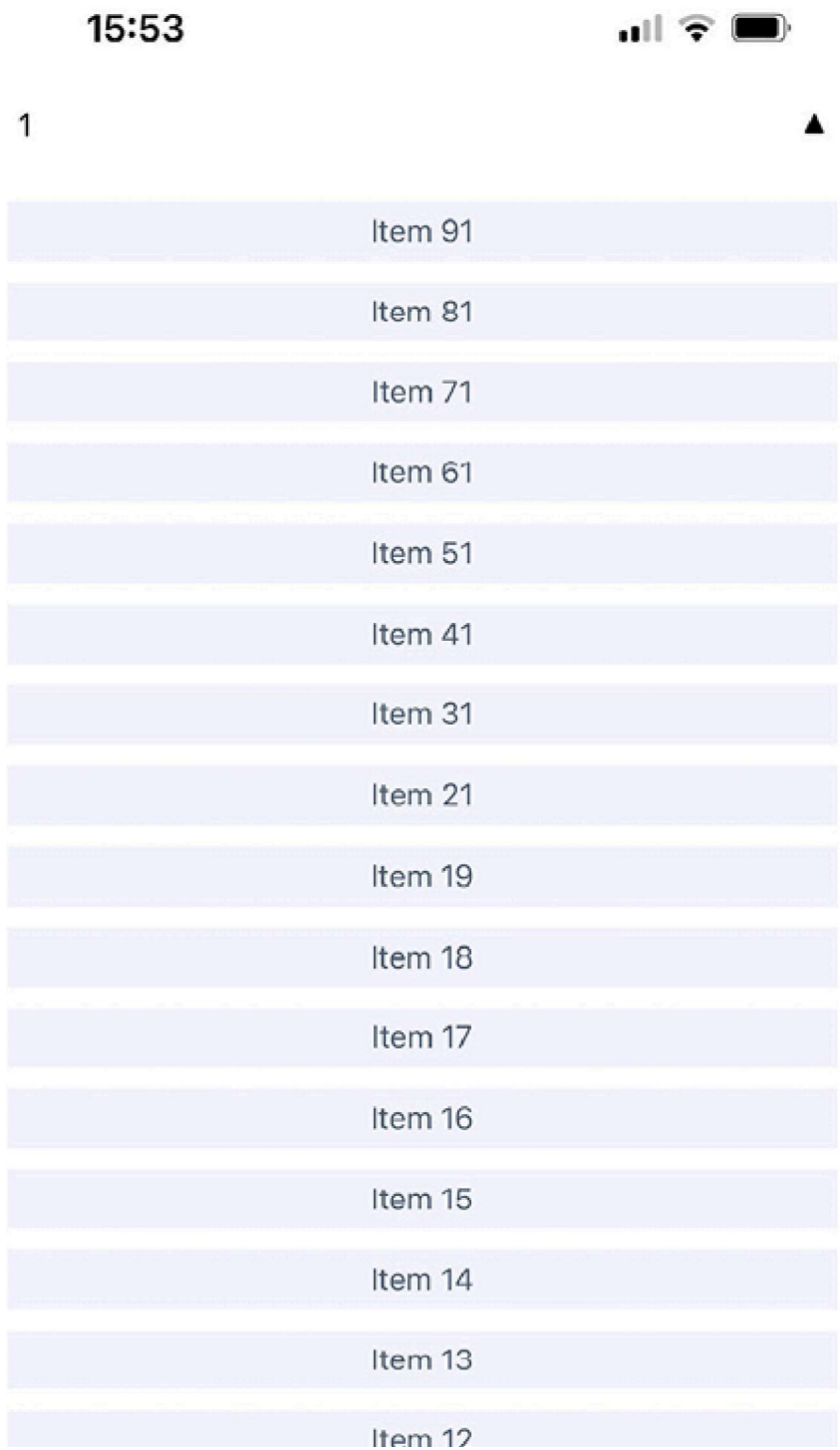

15:04
Search
Item 0
Item 1
Item 10
Item 11
Item 12
Item 13
Item 14
Item 15
Item 16
Item 17
Item 18
Item 19
Item 2
Item 20
Item 21
Item 22

Figure 20.3: The sorting and filtering list

By default, the entire list is rendered in ascending order. You can see the placeholder **Search** text when the user hasn't provided anything yet. Let's see how this looks when you enter a filter and change the sort order:


15:53
1
Item 91
Item 81
Item 71
Item 61
Item 51
Item 41
Item 31
Item 21
Item 19
Item 18
Item 17
Item 16
Item 15
Item 14
Item 13
Item 12

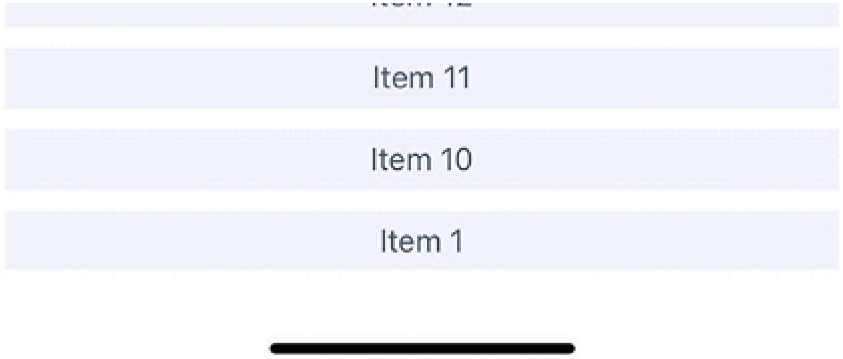


Figure 20.4: The list with a changed sort order and search value

This search includes items containing `1` and sorts the results in descending order. Note that you can either change the order first or enter the filter first. Both the filter and the sort order are part of the `ListContainer` state.

In the next section, you'll learn how to fetch list data from an API endpoint.

Fetching list data

Commonly, you'll fetch your list data from some API endpoint. In this section, you'll learn about making API requests from React Native components. The good news is that the `fetch()` API is polyfilled by React Native, so the networking code in your mobile applications should look and feel a lot like it does in your web applications.

To start things off, let's build a **mock API** for our list items using functions that return promises just like `fetch()` does:

```
const items = new Array(100).fill(null).map((v, i) => `Item ${i}`);
function filterAndSort(data: string[], text: string, asc: boolean) {
  return data
    .filter((i) => text.length === 0 || i.includes(text))
    .sort(
      asc
        ? (a, b) => (b > a ? -1 : a === b ? 0 : 1)
        : (a, b) => (a > b ? -1 : a === b ? 0 : 1)
    );
}
export function fetchItems(
  filter: string,
  asc: boolean
): Promise<{ json: () => Promise<{ items: string[] }> }> {
  return new Promise((resolve) => {
    resolve({
      json: () =>
        Promise.resolve({
          items: filterAndSort(items, filter, asc),
        }),
    });
  });
}
```

With the mock API function in place, let's make some changes to the `ListContainer` component. Instead of using local data

sources, you can now use the `fetchItems()` function to load data from the mock API. Let's take a look and define the `ListContainer` component:

```
export default function ListContainer() {
  const [asc, setAsc] = useState(true);
  const [filter, setFilter] = useState("");
  const [data, setData] = useState<MappedList>([]);
  useEffect(() => {
    fetchItems(filter, asc)
      .then((resp) => resp.json())
      .then(({ items }) => {
        setData(mapItems(items));
      });
  }, []);
```

We've defined state variables using the `useState` and `useEffect` hooks to fetch initial list data.

Now, let's take a look at the usage of our new handlers in the `List` component:

```
  return (
    <List
      data={data}
      asc={asc}
      onFilter={(text) => {
        fetchItems(text, asc)
```

```
            .then((resp) => resp.json())
            .then(({ items }) => {
              setFilter(text);
              setData(mapItems(items));
            });
        }}
        onSort={() => {
          fetchItems(filter, !asc)
            .then((resp) => resp.json())
            .then(({ items }) => {
              setAsc(!asc);
              setData(mapItems(items));
            });
        }}
      />
    );
  }
```

Any action that modifies the state of the list needs to call `fetchItems()` and set the appropriate state once the promise resolves.

In the following section, you'll learn how list data can be loaded lazily.

Lazy list loading

In this section, you'll implement a different kind of list: one that scrolls infinitely. Sometimes, users don't actually know

what they're looking for, so filtering or sorting isn't going to help. Think about the Facebook news feed you see when you log in to your account; it's the main feature of the application, and rarely are you looking for something specific. You need to see what's going on by scrolling through the list.

To do this using a `FlatList` component, you need to be able to fetch more API data when the user scrolls to the end of the list. To get an idea of how this works, you need a lot of API data to work with, and generators are great at this. So, let's modify the mock that you created in the *Fetching list data* section's example so that it just keeps responding with new data:

```
function* genItems() {
  let cnt = 0;
  while (true) {
    yield `Item ${cnt++}`;
  }
}
let items = genItems();
export function fetchItems({ refresh }: { refresh?: boolean }) {
  if (refresh) {
    items = genItems();
  }
  return Promise.resolve({
    json: () =>
      Promise.resolve({
        items: new Array(30).fill(null).map(() => items.next().value as string),
```

```
        }),
    });
  }
```

With `fetchItems`, you can now make an API request for new data every time the end of the list is reached. Eventually, this will fail when you run out of memory, but I'm just trying to show you in general terms the approach you can take to implement infinite scrolling in React Native. Now, let's take a look at what the `ListContainer` component looks like with `fetchItems`:

```
import React, { useState, useEffect } from "react";
import * as api from "./api";
import List from "./List";
export default function ListContainer() {
  const [data, setData] = useState([]);
  function fetchItems() {
    return api
      .fetchItems({})
      .then((resp) => resp.json())
      .then(({ items }) => {
        setData([
          ...data,
          ...items.map((value) => ({
            key: value,
            value,
          })),
```

```
        ]);
      });
  }
  useEffect(() => {
    fetchItems();
  }, []);
  return <List data={data} fetchItems={fetchItems} />;
}
```

Each time `fetchItems()` is called, the response is concatenated with the `data` array. This becomes the new list data source, instead of replacing it as you did in earlier examples.

Now, let's take a look at the `List` component to see how to respond to the end of the list being reached:

```
type Props = {
  data: { key: string; value: string }[];
  fetchItems: () => Promise<void>;
  refreshItems: () => Promise<void>;
  isRefreshing: boolean;
};
export default function List({
  data,
  fetchItems
}: Props) {
  return (
    <FlatList
      data={data}
```

```
        renderItem={({ item }) => <Text style={styles.item}>{item.value}</Text>}
        onEndReached={fetchItems}
      />
    );
  }
```

`FlatList` accepts the `onEndReached` handler prop, which will be invoked every time you reach the end of the list during scrolling.

If you run this example, you'll see that, as you approach the bottom of the screen while scrolling, the list just keeps growing.

Implementing pull to refresh

The **pull-to-refresh** gesture is a common action on mobile devices. It allows users to refresh the content of a view without having to lift a finger from the screen or manually reopen the app, just by pulling it down to trigger a page refresh. Loren Brichter, the creator of Tweetie (later Twitter for iPhone) and Letterpress, introduced this gesture in 2009. This gesture has become so popular that Apple integrated it into its SDKs as `UIRefreshControl`.

To use pull to refresh in the `FlatList` app, we just need to pass a few props and handlers. Let's take a look at our `List`

component:

```
type Props = {
  data: { key: string; value: string }[];
  fetchItems: () => Promise<void>;
  refreshItems: () => Promise<void>;
  isRefreshing: boolean;
};
export default function List({
  data,
  fetchItems,
  refreshItems,
  isRefreshing,
}: Props) {
  return (
    <FlatList
      data={data}
      renderItem={({ item }) => <Text style={styles.item}>{item.value}</Text>}
      onEndReached={fetchItems}
      onRefresh={refreshItems}
      refreshing={isRefreshing}
    />
  );
}
```

As we have provided the `onRefresh` and `refreshing` props, our `FlatList` component automatically enables the pull-to-refresh gesture. The `onRefresh` handler will be called when you

pull the list, and the refreshing property will enable the loading spinner to reflect the loading state.

To apply defined props in the `List` component, let's implement the `refreshItems` function with the `isRefreshing` state in the `ListContainer` component:

```
  const [isRefreshing, setIsRefreshing] = useState(false);
  function fetchItems() {
    return api
      .fetchItems({})
      .then((resp) => resp.json())
      .then(({ items }) => {
        setData([
          ...data,
          ...items.map((value) => ({
            key: value,
            value,
          })),
        ]);
      });
  }
```

In `refreshItems`, as well as in the `fetchItems` method, we get list items but save them as a new list. Also, note that before calling the API, we update the `isRefreshing` state to set it as a `true` value, and in the final block, we set it to `false` to provide information to `FlatList` that loading has ended.

Summary

In this chapter, you learned about the `FlatList` component in React Native. This component is general-purpose, as it doesn't impose any specific look on the items that get rendered. Instead, the appearance of the list is up to you, leaving the `FlatList` component to help with efficiently rendering a data source. The `FlatList` component also provides a scrollable region for the items it renders.

You implemented an example that took advantage of section headers in list views. This is a good place to render static content such as list controls. You then learned about making network calls in React Native; it's just like using `fetch()` in any other web application.

Finally, you implemented lazy lists that scroll infinitely by only loading new items after you've scrolled to the bottom of what's already been rendered. Also, we added a feature to refresh that list by means of a pull gesture.

In the next chapter, you'll learn how to show the progress of network calls, among other things.